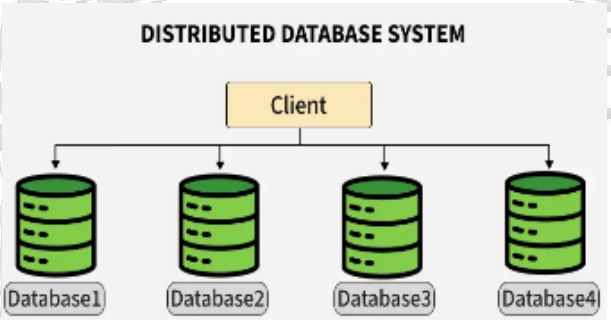
# UNIT-V

## 5.4 Distributed Database System

A Distributed Database System (DDBS) is a collection of multiple databases spread across different physical locations, connected via a network. Unlike a centralized system, where all data is stored in one place, a distributed system manages data across various sites while making it appear as a single database to users. It improves data availability, reliability, and performance by enabling local access, parallel processing, and fault tolerance.




**Types**

Some of the type of distributed database system are:

**1. Homogeneous Database:**

In a homogeneous database, all different sites store database identically. The operating system, database management system, and the data structures used all are the same at all sites. Hence, they're easy to manage.

**Features**:

- Unified query language and interface.
- Low integration complexity.
- Efficient synchronization.

**Example:** A bank with branches in different cities uses Oracle DB at every location. All databases have the same structure and are synchronized regularly.

**2. Heterogeneous Database**

In a heterogeneous distributed database, different sites may use different DBMSs**,** schemas, or data models, making query processing and transactions difficult. Some sites may not even be aware of others, so translation mechanisms are needed for communication.

**Features:**

- Supports interoperability between diverse systems.
- Complex query optimization and transaction management.
- Useful in mergers or collaborations between organizations.

**Example**: A logistics company uses MySQL for inventory, MongoDB for vehicle tracking, and PostgreSQL for billing. Integration middleware allows unified querying across these platforms.

**3. Client-Server Distributed Database System**

In this model, the server stores and manages the database, while clients send queries over the network. It offers centralized control with distributed access, making it ideal for enterprise systems and web applications. Clients can be lightweight while the server handles heavy processing. Example: Web application interacting with a central PostgreSQL server.

**Features:**

- Simplifies resource management.
- Central servers can be optimized for performance.
- Easily scalable with more clients.

**Example**: An e-commerce website where the frontend (client) is hosted separately and interacts with a central PostgreSQL server to manage orders, users, and inventory.

**4. Peer-to-Peer Distributed Database System**

Here, all nodes are equal, with no fixed client or server roles. Each node can store data and also process queries, leading to decentralized control. It supports fault tolerance and high availability. Example: Blockchain networks like Ethereum, where each node maintains a part of the distributed ledger.

**Features:**

- No single point of failure.
- Useful in decentralized and distributed apps.
- High availability and data redundancy.

**Example:** Blockchain-based databases like Ethereum or BitTorrent-based systems, where each peer maintains part of the ledger and participates equally in transactions.

**5. Cloud-Based Distributed Database System**

These systems are deployed on cloud platforms and span multiple geographic regions for scalability and reliability. They abstract infrastructure details and are offered as DBaaS, making them ideal for dynamic workloads. Example: Google Cloud Spanner and Amazon DynamoDB used for global applications.

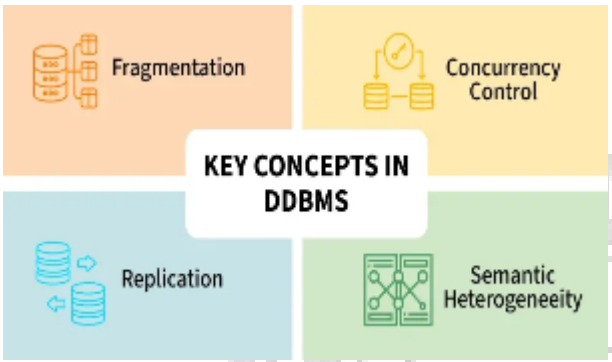
**Features:**

- Automatic scaling and replication.
- Pay-as-you-use pricing.
- Global availability and disaster recovery.

**Example:**

- **Google Cloud Spanner**: Global-scale relational database.
- **Amazon DynamoDB:** Key-value and document database with high performance.
- **Azure Cosmos DB**: Multi-model, globally distributed DBMS.

key components and challenges of a Distributed Database




**1. Replication**

In replication, copies of the same data are stored at two or more sites. If every site has the full database, it's called full replication. This improves data availability and allows faster, parallel query processing. However, updates must be made at all sites, or data may become inconsistent. It also adds overhead and makes concurrency control more complex.

**2. Fragmentation**

In this approach, the relations are fragmented (i.e., they're divided into smaller parts) and each of the fragments is stored in different sites where they're required. It must be made sure that the fragments are such that they can be used to reconstruct the original relation (i.e, there isn't any loss of data).
Fragmentation is advantageous as it doesn't create copies of data, consistency is not a problem.

Fragmentation of relations can be done in two ways:

- **Horizontal fragmentation - Splitting by rows**
  The relation is fragmented into groups of tuples so that each tuple is assigned to at least one fragment.
- **Vertical fragmentation - Splitting by columns**
  The schema of the relation is divided into smaller schemas. Each fragment must contain a common candidate key so as to ensure a lossless join.

In certain cases, an approach that is hybrid of fragmentation and replication is used.

**3. Concurrency Control**

Concurrency control ensures data remains accurate when multiple transactions run at the same time. Without it, issues like lost updates or dirty reads can occur. Its goal is to make parallel transactions behave as if run one by one. Common methods include locking, timestamps, and optimistic concurrency.

**4.Semantic Heterogeneity**

Semantic heterogeneity happens when different databases use the same data labels but with different meanings, formats, or units. For example, one system may store salary in

dollars, another in rupees. This can cause confusion during data integration, so resolving it is important for accurate results.

**Functions of Distributed Database:**

- It is used in Corporate Management Information System.
- It is used in multimedia applications.
- Used in Military's control system, Hotel chains etc.
- It is also used in manufacturing control system.

**Advantages of Distributed Database System :**

- There is fast data processing as several sites participate in request processing.
- Reliability and availability of this system is high.
- It possess reduced operating cost.
- It is easier to expand the system by adding more sites.
- It has improved sharing ability and local autonomy.

**Disadvantages of Distributed Database System :**

- The system becomes complex to manage and control.
- The security issues must be carefully managed.
- The security issues must be carefully managed.
- The system require deadlock handling during the transaction processing otherwise
- The entire system may be in inconsistent state.
- There is need of some standardization for processing of distributed database system.